

S1/S2 分割模型总结说明

更新日期：2026-07-07

本文用于解释 `work_space/S1` 和 `work_space/S2` 两个分割模型部分目前分别做什么、输入输出是什么、如何和 G1/G2 配合、怎么训练与调参，以及当前结果进度。本文只总结当前仓库里的真实代码和文档状态，不把尚未跑出的训练结果写成已完成。

1. 总体定位

S1 和 S2 同属分割模型小组，最终目标是把真实数据和 G1/G2 提供的合成数据转化为可量化的分割模型结果，用来回答两个问题：

- 1. 真实数据基线能做到什么水平。
- 2. 加入 G1 生成、G2 QC 通过的 synthetic data 后，分割性能是否提高，尤其是否改善 RC、ET、TC、WT 和小病灶指标。

当前两条线的定位不同：

模块	当前代码位置	技术路线	作用	当前成熟度
S1	<code>work_space/S1/brats2026_multitask_S1_v2/repository</code>	MONAI 3D UNet 多任务模型	同时预测 tumor subregions 和 RC，作为探索性多任务方案	已有代码框架，暂无训练结果
S2	<code>work_space/S2/BraTS2026_S2_RC_v1.0/repository</code>	nnU-Net v2 + 自定义 trainer	作为主基线和 RC 优化线，更接近正式训练流程	代码更接近可训练主线，暂无训练结果

简单说：

- S2 应优先作为主力 baseline 跑通，因为它基于 nnU-Net v2，和官方分割任务更贴近。
- S1 可作为多任务探索和辅助对照，重点看 RC head 是否能带来额外收益。

2. 和 G1/G2 的协作关系

四个模块之间的数据流应该按下面顺序走：

```

原始 Task1 数据
|
v
G2 真实数据 QC、manifest、固定 train/val/test split
|
+--> S1/S2 real-only baseline 训练
|
+--> G1 T2W completion 或 G1 diffusion augmentation
      |
      v
      G2 synthetic intake + QC + accepted/rejected
      |
      v
      G2 物化 real+synth 训练集
      |
      v
      S1/S2 real+synth 消融训练
      |
      v
      G2 解析指标、汇总 ablation report

```

2.1 G1 给 S1/S2 提供什么

G1 目前有两类输出，不能混淆：

1. T2W completion

- 输入：缺失或 fake/broken T2W 的病例，通常含 t1n/t1c/t2f/seg，不含可用 t2w。
- 输出：补出的 <case_id>-t2w.nii.gz。
- 用途：修复真实病例中的缺失/异常 T2W。它默认不是新增 synthetic case，而是补齐原病例通道。

2. Diffusion augmentation

- 输入：完整真实病例的 seg 或 lesion/ROI 条件。
- 输出：synthetic t1c/t1n/t2w/t2f/seg。
- 用途：作为额外 synthetic cases 追加到训练集，用于 real+synth augmentation。

2.2 G2 给 S1/S2 提供什么

G2 的职责是把 G1 的 raw output 变成可训练、可追溯、可审计的数据资产。

S1/S2 不应该直接吃 G1 raw output，而应该吃 G2 产物：

- 真实数据 manifest：
work_space/G2/results/manifests/real_train_manifest.csv
- 固定划分：
work_space/G2/results/splits/splits_final_train_val_test.json

- nnU-Net real-only 入口：
work_space/G2/results/nnunet_raw/Dataset260_BraTS2026_MET_RealOnly/
- synthetic accepted/rejected manifest: 正式 G1 run 后由 G2 生成
- real+synth nnU-Net 物化入口: 正式 G1 run 后由 G2 生成
- QC 报告和 ablation 模板: work_space/G2/results/qc/、
work_space/G2/results/reports/

2.3 当前统一通道顺序

S1/S2 已统一为 G2 物化数据和 nnU-Net dataset.json 默认顺序:

```
0000 = t1n
0001 = t1c
0002 = t2w
0003 = t2f
```

具体体现:

- S1 loader 和 S1 inference 读取顺序是 t1n, t1c, t2w, t2f。
- S2 01_convert_to_nnunet.py 写出顺序是 0000=t1n, 0001=t1c, 0002=t2w, 0003=t2f。
- S2 dataset.json 的 channel_names 是 0=T1N, 1=T1C, 2=T2W, 3=T2F。

后续 G2 物化 real-only 和 real+synth 数据时必须沿用这一顺序。不要再使用旧顺序 t1c, t1n, t2f, t2w。

3. S1 说明

3.1 S1 的作用

S1 是一个 MONAI 多任务模型框架，目标是在一个模型里同时完成两个任务:

1. tumor subregion segmentation
 - 输出 4 类: background、NETC、SNFH、ET。
2. RC segmentation
 - 输出 2 类: background、RC。

设计思路是共享一个 3D UNet backbone，再接两个输出 head:

- tumor head: 预测 tumor subregions。
- RC head: 预测 recurrence component。

这个方案的价值在于探索 RC 是否应该作为单独任务显式学习，而不是只作为普通多类分割中的一个类别。

3.2 S1 当前代码结构

```
work_space/S1/brats2026_multitask_S1_v2/repository/  
  configs/  
    multitask_v1.yaml  
    multitask_v1_full.yaml  
  datasets/  
    brats_multitask_dataset.py  
  models/  
    multitask_unet.py  
  losses/  
    multitask_loss.py  
    dice_metric.py  
  trainers/  
    trainer_v1_final.py  
  inference/  
    infer_multitask.py  
  scripts/  
    01_apply_corrected_labels.py  
    02_find_invalid_labels.py  
    03_fix_invalid_labels.py  
    08_build_full_multitask_labels.py  
    09_dataset_audit.py  
    10_create_full_split.py
```

3.3 S1 输入

S1 训练输入是病例目录，每个病例目录至少需要：

```
<case_id>-t1n.nii.gz  
<case_id>-t1c.nii.gz  
<case_id>-t2w.nii.gz  
<case_id>-t2f.nii.gz  
tumor_label.nii.gz  
rc_label.nii.gz
```

其中：

- tumor_label.nii.gz 由原始 seg 去掉 RC 后生成，标签为 0/1/2/3。
- rc_label.nii.gz 由原始 seg 中 label 4 提取而来，标签为 0/1。

生成方式：

```
cd work_space/S1/brats2026_multitask_S1_v2/repository  
python scripts/08_build_full_multitask_labels.py  
python scripts/09_dataset_audit.py  
python scripts/10_create_full_split.py
```

路径现在支持环境变量或命令行参数。推荐服务器先统一设置：

```
export BRATS_TRAIN_ROOT=/path/to/MICCAI-LH-BraTS2025-MET-Challenge-Training
export BRATS_CORRECTED_ROOT=/path/to/MICCAI-LH-BraTS2025-MET-Challenge-corrected-labels
export BRATS_SPLIT_DIR=/path/to/splits
```

也可以在单个脚本里显式传参：

```
python scripts/08_build_full_multitask_labels.py --train-root "$BRATS_TRAIN_ROOT"
python scripts/09_dataset_audit.py --train-root "$BRATS_TRAIN_ROOT"
python scripts/10_create_full_split.py --train-root "$BRATS_TRAIN_ROOT" --split-dir "$BRATS_SPLIT_DIR"
```

3.4 S1 输出

训练输出：

```
checkpoints_full/latest.pth
checkpoints_full/best.pth
tensorboard_full/
```

推理输出：

```
tumor_pred.nii.gz
rc_pred.nii.gz
```

其中 S1 推理当前会分别保存 tumor 和 RC 两个结果。若要变成最终 BraTS 多类提交格式，还需要增加一个融合步骤：

```
final_seg = tumor_pred
final_seg[rc_pred == 1] = 4
```

并且必须检查 label 合法范围只包含 0,1,2,3,4。

3.5 S1 训练命令

全量训练入口：

```
cd work_space/S1/brats2026_multitask_S1_v2/repository
python trainers/trainer_v1_final.py \
    --config configs/multitask_v1_full.yaml
```

当前 full config 的核心参数：

```
epochs: 300
batch_size: 2
lr: 0.0003
patch_size: [96, 96, 96]
amp: true
```

首次训练要特别注意：

- configs/multitask_v1_full.yaml 里 train.resume 已默认设为空字符串。
- 第一次训练可以直接跑；需要续训时再设置 resume，或者用环境变量 S1_RESUME=/path/to/latest.pth 覆盖。

```
train:  
  resume: ""
```

3.6 S1 推理命令

```
cd work_space/S1/brats2026_multitask_S1_v2/repository  
python inference/infer_multitask.py \  
  --checkpoint /path/to/best.pth \  
  --case_dir /path/to/BraTS-MET-xxxxx-xxx \  
  --output_dir /path/to/output_case
```

推理脚本已改成根据自身文件位置自动定位 repository，不再依赖固定服务器路径。

3.7 S1 调参重点

优先调参顺序：

1. patch_size

- 当前是 96,96,96。
- 显存不足先降 batch size，不要轻易把 patch 降太小。

2. batch_size

- 当前 full config 是 2。
- 如果 24G GPU OOM，先试 1。

3. lr

- 当前是 $3e-4$ 。
- 如果 loss 震荡或 NaN，先降到 $1e-4$ 。

4. RC loss 权重或不确定性权重策略

- 当前 MultiTaskLoss 使用 uncertainty weighting。
- 如果 RC dice 长期为 0，可考虑显式给 RC loss 加权。

5. tumor/RC 融合策略

- S1 的 RC 是单独 head，最后如何覆盖 tumor_pred 会影响最终 label map。

- 推荐先使用 `rc_pred==1` 覆盖为 4，再做 connected component 和小病灶检查。

3.8 S1 当前结果状态

当前仓库中：

- 有多任务模型代码。
- 有 sample50 pilot 文档，记录数据准备完成。
- 没有提交训练 checkpoint。
- 没有提交 Dice、NSD 或官方风格 metrics。
- `work_space/S1/results/` 仍为空。

所以 S1 当前应视为“代码框架已提交，训练结果未提交”。

4. S2 说明

4.1 S2 的作用

S2 是当前更接近主线 baseline 的实现，基于 nnU-Net v2。它的目标是：

1. 把 BraTS-MET 数据转换成 nnU-Net raw dataset。
2. 使用固定 train/val split。
3. 使用自定义 trainer 强化 RC 类别学习。
4. 训练 real-only baseline。
5. 后续训练 real+synth ablation，对比 synthetic data 是否有效。

4.2 S2 当前代码结构

```
work_space/S2/BraTS2026_S2_RC_v1.0/repository/  
  scripts/  
    01_apply_corrected_labels.py  
    01_convert_to_nnunet.py  
    02_find_invalid_labels.py  
    03_fix_invalid_labels.py  
  custom_nnunet/  
    nnUNetTrainerBraTS2026RC.py  
  train.sh  
  infer.sh  
  docs/  
    DATASET.md  
    TRAINING.md  
    INFERENCE.md  
    TUNING.md
```

注意：

- `train.py` 当前是空文件。

- inference.py 当前是空文件。
- 实际训练入口是 train.sh。
- 实际推理入口是 infer.sh。

4.3 S2 输入

S2 原始输入是 BraTS-MET 病例目录，每个病例目录至少需要：

```
<case_id>-t1n.nii.gz  
<case_id>-t1c.nii.gz  
<case_id>-t2w.nii.gz  
<case_id>-t2f.nii.gz  
<case_id>-seg.nii.gz
```

S2 会转换为 nnU-Net raw 格式：

```
nnUNet_raw/  
  Dataset501_BraTS2026Tumor/  
    imagesTr/  
      <case_id>_0000.nii.gz  
      <case_id>_0001.nii.gz  
      <case_id>_0002.nii.gz  
      <case_id>_0003.nii.gz  
    labelsTr/  
      <case_id>.nii.gz  
  dataset.json
```

当前转换脚本的通道顺序是：

```
0000 = t1n  
0001 = t1c  
0002 = t2w  
0003 = t2f
```

这必须和 G2 物化数据保持一致。

4.4 S2 数据转换命令

```
cd work_space/S2/BraTS2026_S2_RC_v1.0/repository  
python scripts/01_apply_corrected_labels.py  
python scripts/02_find_invalid_labels.py  
python scripts/03_fix_invalid_labels.py  
python scripts/01_convert_to_nnunet.py
```

推荐服务器先设置：


```
export BRATS_TRAIN_ROOT=/path/to/MICCAI-LH-BraTS2025-MET-Challenge-Training
export BRATS_CORRECTED_ROOT=/path/to/MICCAI-LH-BraTS2025-MET-Challenge-corrected-labels
export BRATS_SPLIT_DIR=/path/to/splits
export nnUNet_raw=/path/to/nnunet_raw
export nnUNet_preprocessed=/path/to/nnunet_preprocessed
export nnUNet_results=/path/to/nnunet_results
```

正式服务器运行前要确认：

1. 源数据路径存在。
2. 01_convert_to_nnunet.py 会自动创建 imagesTr/ 和 labelsTr/。
3. dataset.json 中 labels 为 background, NETC, SNFH, ET, RC。
4. train/val split 文件存在。

4.5 S2 训练命令

```
cd work_space/S2/BraTS2026_S2_RC_v1.0/repository
bash train.sh
```

train.sh 设置：

```
export nnUNet_raw="${nnUNet_raw:-${REPO_DIR}/data/nnunet_raw}"
export nnUNet_preprocessed="${nnUNet_preprocessed:-${REPO_DIR}/data/nnunet_preprocessed}"
export nnUNet_results="${nnUNet_results:-${REPO_DIR}/data/nnunet_results}"
export BRATS_SPLIT_DIR="${BRATS_SPLIT_DIR:-${REPO_DIR}/data/splits}"
export nnUNet_extTrainer="${REPO_DIR}/custom_nnunet"
export PYTHONPATH="${REPO_DIR}:${PYTHONPATH}"

nnUNetv2_train 501 3d_fullres 0 \
  -tr nnUNetTrainerBraTS2026RC \
  -num_gpus 1
```

训练前还需要执行标准 nnU-Net 预处理流程，当前仓库文档没有完整写出，但实际应该包括：

```
nnUNetv2_plan_and_preprocess -d 501 --verify_dataset_integrity
```

然后再运行 `bash train.sh`。

4.6 S2 推理命令

```
cd work_space/S2/BraTS2026_S2_RC_v1.0/repository
bash infer.sh INPUT_FOLDER OUTPUT_FOLDER
```

脚本内部调用：

```
nnUNetv2_predict \  
-i ${INPUT_FOLDER} \  
-o ${OUTPUT_FOLDER} \  
-d 501 \  
-c 3d_fullres \  
-f 0
```

输出应是 nnU-Net 风格的 multi-class segmentation NIfTI 文件，标签应包含：

```
0 = background  
1 = NETC  
2 = SNFH  
3 = ET  
4 = RC
```

4.7 S2 自定义 trainer 做了什么

核心文件：

```
work_space/S2/BraTS2026_S2_RC_v1.0/repository/custom_nnunet/nnUNetTrainerBraTS2026RC  
.py
```

主要改动：

1. 固定训练轮数
 - num_epochs = 1000
2. RC-aware CE 权重
 - CE weight = [1.0, 1.0, 1.0, 1.0, 3.0]
 - RC 类别权重为 3。
3. 固定 split
 - train_full.txt
 - val_full.txt
4. 保留 nnU-Net v2 原始训练、验证、deep supervision、augmentation 和推理框架。

4.8 S2 调参重点

优先调参顺序：

1. RC CE weight
 - 当前是 3。
 - 可试：3、5、10。
 - 如果 RC recall 太低，提高权重。
 - 如果 false positive 太多，降低权重或加后处理。

2. patch size / batch size

- 由 nnU-Net plan 决定。
- 如果显存不足，优先让 nnU-Net 自动 plan，而不是手工硬改。

3. epoch 数

- 当前 1000。
- 如果过拟合，观察 validation Dice/NSD 后早停或选 best checkpoint。

4. foreground oversampling

- 当前 oversample_foreground_percent = 0.33。
- 小病灶漏检多时，可考虑提高。

5. post-processing

- 对 tiny false positive 可用连通域过滤。
- 但不能一刀切删除小病灶，因为官方榜单包含 small instance 指标，小病灶是真正评分重点。

4.9 S2 当前结果状态

当前仓库中：

- 有 nnU-Net v2 数据转换脚本。
- 有自定义 trainer。
- 有 train/infer shell 入口。
- 有训练和推理说明文档。
- 没有提交 checkpoint。
- 没有提交 validation Dice、NSD 或官方风格 metrics。
- work_space/S2/results/ 仍为空。

所以 S2 当前应视为“主线训练代码已提交，训练结果未提交”。

5. real-only 与 real+synth 实验设计

S1/S2 后续必须至少跑两组实验：

实验	训练数据	验证数据	目的
real-only baseline	G2 final QC pass 的真实 train	固定 real val	得到无 synthetic 的基准
real+synth smoke	real train + 少量 accepted synthetic	固定 real val	检查 synthetic 是否不会拉坏性能
real+synth main	real train + 策略比例 accepted synthetic	固定 real val	正式评估 G1/G2 synthetic augmentation

实验	训练数据	验证数据	目的
			是否有效
final internal test	最优方案	固定 internal test	只做最终内部评估，不调参

不能用 internal test 调参。官方 validation/pseudo-test 也不能用于模型选择。

6. 结果应该怎么汇总

S1/S2 训练结束后，至少要交给 G2:

1. 模型配置

- 代码 commit。
- config。
- 数据版本。
- channel order。
- train/val split。
- synthetic run id，如果使用 synthetic。

2. checkpoint 信息

- best checkpoint。
- final checkpoint。
- 训练轮数。
- 是否 resume。

3. 验证集预测结果

- 每个 case 的 predicted segmentation。
- 原始 label。
- inference 命令。

4. 指标表

- Dice。
- NSD。
- lesion-wise DSC/NSD。
- small instance TP/FN/FP/F1。
- ET、TC、WT、RC 分别统计。

5. 消融结论

- real-only vs real+synth。
- 是否提升 RC。

- 是否提升小病灶。
- 是否拉低 ET/TC/WT。
- 如果只对某类病灶有效，结论必须限制使用范围。

7. 当前缺口和下一步

7.1 当前缺口

1. S1/S2 都还没有提交实际训练结果。
2. S1/S2 顶层 README 仍是“待填写”。
3. S1/S2 的主要脚本已支持环境变量或命令行参数，但服务器运行前仍需要操作者明确设置数据目录和 nnU-Net 目录。
4. S2 的 requirements.txt 是空文件。
5. S2 的 train.py 和 inference.py 是空文件，容易误导操作者。
6. S1 当前推理输出不是最终单文件 label map，还需要融合 tumor/RC。
7. 真实数据和 synthetic 数据的 real-only / real+synth 消融还没有跑。

7.2 建议下一步顺序

1. 先在服务器设置统一路径。
 - BRATS_TRAIN_ROOT 指向真实训练数据。
 - BRATS_CORRECTED_ROOT 指向 corrected labels。
 - BRATS_SPLIT_DIR 指向固定 split 文件目录。
 - nnUNet_raw、nnUNet_preprocessed、nnUNet_results 指向服务器希望使用的 nnU-Net 工作目录。
2. 先跑 S2 real-only。
 - 先转换 nnU-Net raw。
 - 跑 nnUNetv2_plan_and_preprocess。
 - 跑 `bash train.sh`。
 - 用固定 val 做指标。
3. 再跑 S1 sample/full。
 - 先确认 resume 为空或指向真实 checkpoint。
 - 再确认 BRATS_TRAIN_ROOT、BRATS_TRAIN_SPLIT、BRATS_VAL_SPLIT 或 config 相对路径可访问。
 - 跑 sample smoke。
 - 再跑 full config。
4. 等 G1 输出 synthetic 后，由 G2 先 QC。

- 没有 G2 accepted manifest, 不要把 G1 raw synthetic 直接交给 S1/S2。

5. 跑 real+synth 消融。

- 只改变训练集。
- val/test 固定不动。
- 指标由 G2 统一解析和归档。

8. 一句话结论

S2 是当前优先跑通的 nnU-Net v2 主基线；S1 是多任务 RC 探索线。G1 负责生成补全或 synthetic 数据，G2 负责 QC、manifest、物化和指标汇总。S1/S2 不能直接吃 G1 raw output，必须吃 G2 放行后的 real-only 或 real+synth 数据入口。当前仓库已有 S1/S2 代码框架，通道顺序已统一为 t1n,t1c,t2w,t2f，主要路径已改成环境变量/参数可配置，但还没有提交可证明模型效果的训练结果和官方风格指标。